Cairo University
Faculty of Engineering

Department of Computer
Engineering

Secure Stream Cipher System Using One-Time Pad, Key Exchange, and Message Authentication

Cryptographic course

Submitted to

Eng.Salma

Submitted by Mohamed Abdelaziz

Name	Sec	BN
Mazen Adel Abdallah	2	47
Mohmmed Abdelaziz Khater	2	54

Project Overview

This project implements a secure communication system using a stream cipher based on the One-Time Pad (OTP) methodology. The system ensures confidentiality, integrity, and authenticity through the use of Diffie-Hellman key exchange, symmetric encryption for seed transmission, and HMAC for authentication. Communication is handled through modular client-server components in Python.

Contents

Communication Flow

Client Side

- Performs Diffie-Hellman key exchange.
- Encrypts and authenticates the seed.
- Sends encrypted file chunks using the stream cipher.

Server Side

- Completes the key exchange and derives shared key.
- Decrypts and verifies the seed.
- Receives encrypted file chunks and writes to output.

Code

- Refer to the client & server files as it is too big to be included here.

Test Case

- Input and output file

key_exchange

Purpose : Implements the Diffie-Hellman key exchange protocol.

Algorithm Details

Both parties agree on a large prime number **p** and a generator **g**.

1. Each party generates a random private key:
    - Sender: **private_key_A**
    - Receiver: **private_key_B**
2. Each party computes their public key:
    a. Sender: $\text{public}_{\text{key}_A} = \text{g}^{\{\text{private}_{\text{key}_A}\}} \bmod \text{p}$
    b. $Receiver: public_{key_b} = g^{\{private_{key_b}\}} mod p$
3. The parties exchange their public keys.
4. Each party computes the shared secret:
    a. $Sender: shared_{secret_A} = public_{key_B}^{\{private_{key_A}\}} \ mod \ p$
    b. $Receiver: shared_{secret_B} = public_{key_A}^{\{private_{key_B}\}} \ mod \ p$
    c. Due to the properties of modular exponentiation, both parties arrive at the same shared secret:*shared_secretA =shared_secretB*

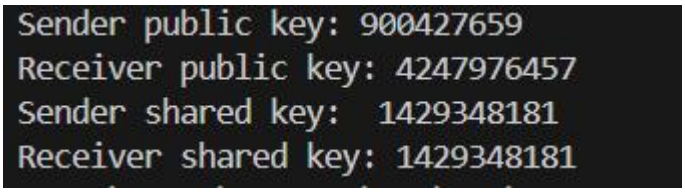
Design Choices

Standard values for the base and the prime number from RFC 3526 (Group 14)

Implementation with a test case:

```python
import random

# 2048-bit MODP Group from RFC 3526 (Group 14)
MODP_2048_P = int("""
FFFFFFFFFFFFFFFFC90FDAA22168C234C4C6628B80DC1CD129024E08
8A67CC74020BBEA63B139B22514A08798E3404DD
EF9519B3CD3A431B302B0A6DF25F14374FE1356D
6D51C245E485B576625E7EC6F44C42E9A63A3620
FFFFFFFFFFFFFFFF
""".replace("\n", "").replace(" ", ""), 16)

MODP_2048_G = 2

class DiffieHellman:
    def __init__(self, p=None, g=None):
        self.p = p or MODP_2048_P
        self.g = g or MODP_2048_G
        self.private_key = random.randint(2, self.p - 2)
        self.public_key = pow(self.g, self.private_key, self.p)

    def generate_shared_key(self, other_public_key):
        shared_secret = pow(other_public_key, self.private_key, self.p)
        return shared_secret

if __name__ == "__main__":
    # Sender side
    sender = DiffieHellman()
    print("Sender public key:", sender.public_key)

    # Receiver side
    receiver = DiffieHellman(p=sender.p, g=sender.g)
    print("Receiver public key:", receiver.public_key)

    # Exchange and generate shared keys
    sender_shared = sender.generate_shared_key(receiver.public_key)
    receiver_shared = receiver.generate_shared_key(sender.public_key)

    print("Sender shared key: ", sender_shared)
    print("Receiver shared key:", receiver_shared)

    assert sender_shared == receiver_shared, "Shared keys don't match!"

```

Test Case output



Seed Encryption – AES-CBC

Purpose : Encrypts the seed used by the stream cipher to ensure it remains confidential during transmission.

Key Components

- Key Derivation : Derives a 256-bit AES key from a shared secret using SHA-256.
- AES Encryption : Uses AES in CBC mode with PKCS7 padding to encrypt the seed.
- Initialization Vector (IV) : Generates a random IV for each encryption to ensure uniqueness.

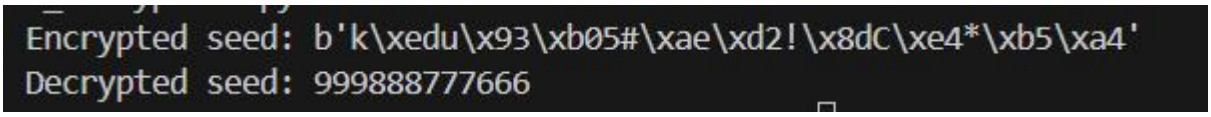
Algorithm Details :

- Encryption :
    - Convert the seed to bytes.
    - Pad the seed to match the AES block size (16 bytes) using PKCS7 padding.
    - Generate a random IV.
    - Encrypt the padded seed using AES in CBC mode.
- Decryption :
    - Decrypt the ciphertext using AES in CBC mode.
    - Unpad the decrypted data to retrieve the original seed.

Implementation with a test case:

```python
import os
import hashlib
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives import padding
from cryptography.hazmat.backends import default_backend

class SeedEncryptor:
    def __init__(self, shared_secret):
        # Derive a 256-bit AES key from shared secret using SHA-256
        self.key = hashlib.sha256(str(shared_secret).encode()).digest()

    def encrypt(self, seed: int) -> tuple[bytes, bytes]:
        seed_bytes = str(seed).encode('utf-8')

        # Pad seed to AES block size (16 bytes)
        padder = padding.PKCS7(128).padder()
        padded_seed = padder.update(seed_bytes) + padder.finalize()

        iv = os.urandom(16)
        cipher = Cipher(algorithms.AES(self.key), modes.CBC(iv), backend=default_backend())
        encryptor = cipher.encryptor()
        ciphertext = encryptor.update(padded_seed) + encryptor.finalize()
        return iv, ciphertext

    def decrypt(self, iv: bytes, ciphertext: bytes) -> int:
        cipher = Cipher(algorithms.AES(self.key), modes.CBC(iv), backend=default_backend())
        decryptor = cipher.decryptor()
        padded_seed = decryptor.update(ciphertext) + decryptor.finalize()

        # Unpad
        unpadder = padding.PKCS7(128).unpadder()
        seed_bytes = unpadder.update(padded_seed) + unpadder.finalize()
        return int(seed_bytes.decode('utf-8'))


if __name__ == "__main__":
    shared_secret = 1234567890987654321  # Pretend this came from DH

    seed_encryptor = SeedEncryptor(shared_secret)
    seed = 999888777666

    iv, encrypted = seed_encryptor.encrypt(seed)
    print("Encrypted seed:", encrypted)

    decrypted = seed_encryptor.decrypt(iv, encrypted)
    print("Decrypted seed:", decrypted)

    assert decrypted == seed, "Seed mismatch!"

```

Test Case output:



Seed Authentication

Purpose : Authenticates the encrypted seed to ensure its integrity and prevent tampering.

Key Components :

- Key Derivation : Derives a 256-bit HMAC key from a shared secret using SHA-256.
- HMAC Generation : Computes an HMAC tag for the encrypted seed using HMAC-SHA256.
- Verification : Verifies the HMAC tag to ensure the encrypted seed has not been altered.

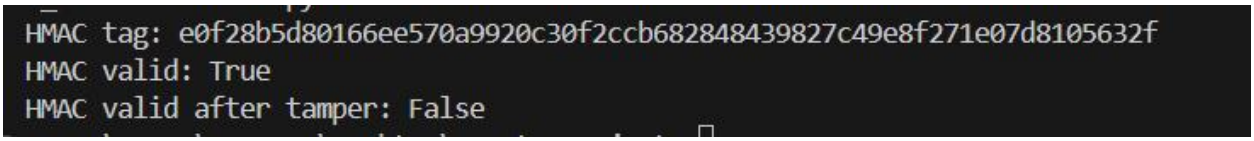
Algorithm Details :

- HMAC Generation :
  - Derive a key using SHA-256 hash of the shared secret.
  - Compute the HMAC tag for the encrypted seed using the derived key.
- Verification :
  - Recompute the expected HMAC tag for the received encrypted seed.
  - Compare the received HMAC tag with the recomputed tag using a constant-time comparison.

Implementation with a test case:

```python
import hmac
import hashlib

class SeedAuthenticator:
    def __init__(self, shared_secret):
        # Derive a 256-bit key for HMAC
        self.key = hashlib.sha256(str(shared_secret).encode()).digest()

    def generate_hmac(self, message: bytes) -> bytes:
        return hmac.new(self.key, message, hashlib.sha256).digest()

    def verify_hmac(self, message: bytes, received_hmac: bytes) -> bool:
        expected_hmac = self.generate_hmac(message)
        return hmac.compare_digest(expected_hmac, received_hmac)

if __name__ == "__main__":
    shared_secret = 1234567890987654321  # Simulated DH shared secret

    authenticator = SeedAuthenticator(shared_secret)

    message = b"Encrypted seed here"
    tag = authenticator.generate_hmac(message)
    print("HMAC tag:", tag.hex())

    # Receiver side verification
    is_valid = authenticator.verify_hmac(message, tag)
    print("HMAC valid:", is_valid)

    # Try tampering
    fake = b"Tampered message"
    is_valid_fake = authenticator.verify_hmac(fake, tag)
    print("HMAC valid after tamper:", is_valid_fake)

```

Test Case output:



Linear Congruential Generator

Purpose : Generates a pseudorandom keystream for the stream cipher.

Key Components :

- Parameters : Multiplier (**a**), increment (**c**), modulus (**m**), and initial state (**seed**).
- Keystream Generation : Produces a sequence of pseudorandom numbers modulo 256 for use as a keystream.

Algorithm Details :

- Next State :$X_{n+1} = (a \cdot X_n + c) \bmod m$
- Keystream Generation :
  - Generate a sequence of pseudorandom numbers using the LCG formula.
  - Take each number modulo 256 to ensure it fits within a single byte.

Design Choices:

Following Park–Miller Parameters I choose a,c,m according.

Implementation with a test case:

```python
class LCG:
    def __init__(self, seed, a=16807, c=0, m=2**31 - 1):
        self.a = a
        self.c = c
        self.m = m
        self.state = seed % self.m  # ensure state is within modulus

    def next(self):
        self.state = (self.a * self.state + self.c) % self.m
        return self.state

    def get_keystream(self, length):
        return [self.next() % 256 for _ in range(length)]  # 1-byte output

if __name__ == "__main__":
    seed = 123456789
    lcg = LCG(seed)
    keystream = lcg.get_keystream(10)
    print("Generated keystream:", keystream)

```

Test Case output:

```
Generated keystream: [112, 15, 34, 25, 164, 179, 118, 93, 24, 151]
```